A Novel Framework for Machine Learning-Based Attack Classification on Bluetooth Low Energy Gateways

The Bluetooth Gateway as a New Security Frontier

The proliferation of Internet of Things (IoT) devices has fundamentally reshaped the landscape of personal and enterprise networks. Central to this transformation is Bluetooth Low Energy (BLE), a wireless communication protocol engineered for short-range, low-power applications.[1] Its efficiency and low cost have driven its integration into a vast array of critical systems, extending far beyond simple consumer electronics. Today, BLE is the de facto standard for Wireless Body Area Networks (WBANs) that collect and transmit sensitive health data, such as heart rate and glucose levels.[2] It is also integral to physical security systems, including residential smart locks, automotive keyless entry systems, and corporate building access control.[3] This widespread adoption in mission-critical and life-critical applications has created an expansive attack surface, exposing consumers and organizations to significant security, privacy, and safety risks.[4]

Systemic Vulnerabilities in the BLE Standard and Implementations

The security challenges inherent in BLE are not merely incidental; they are deeply rooted in its original design philosophy. The protocol was initially conceived for short-distance connections, where security was considered a secondary concern to power efficiency and ease of use.[1] This has resulted in a "security afterthought" architecture that, while improved in subsequent versions (4.1, 4.2, 5.0), still contains foundational vulnerabilities. A critical factor exacerbating this issue is the governance model of the Bluetooth Special Interest Group (SIG). The Bluetooth SIG recommends a suite of security features—including various association models, encryption, and authentication—but does not mandate their full implementation by manufacturers.[4]

This policy has led to a fragmented and inconsistent security landscape, where the actual level of protection afforded to a user is left to the discretion of individual product vendors.

This implementation gap has been the source of numerous documented vulnerabilities. For instance, while the BLE specification calls for a six-digit passkey for pairing, some manufacturers have been found to use a shortened four-digit PIN, drastically reducing the complexity and making brute-force attacks trivial to execute.[4] More systemic flaws exist at the protocol level itself. The legacy pairing method in Bluetooth 4.0 is susceptible to offline brute-force attacks due to low entropy in the passkey.[1] Furthermore, the negotiation of security capabilities during the pairing process is unauthenticated. This allows an attacker to perform a downgrade attack by intercepting and modifying pairing messages, forcing two devices that would normally use a secure method like Numeric Comparison (NC) to fall back to the insecure "Just Works" (JW) model, thereby gaining access without user knowledge.[1] The Key Negotiation of Bluetooth (KNOB) attack exploits a similar vulnerability, allowing an attacker to force the negotiation of an encryption key with very low entropy (e.g., a single byte), which can then be easily cracked.[6] These protocol-level issues are compounded by vendor-specific implementation flaws. Analysis of public vulnerability disclosures (CVEs) reveals that a substantial percentage of Bluetooth-related security issues are tied to specific vendor stacks, such as Google's Android, the Linux BlueZ stack, and chipsets from MediaTek and Qualcomm.[7] This indicates that the software implementing the BLE protocol is as significant an attack surface as the protocol specification itself.

The Gateway: A Centralized Point of Aggregated Risk

In a typical IoT deployment, numerous disparate BLE devices—sensors, actuators, wearables, and locks—do not communicate directly with the internet but connect to a central gateway. This gateway, which may be a dedicated hardware device, a smartphone, or a smart home hub, acts as a bridge, translating BLE communications to other protocols like Wi-Fi or Ethernet. As it aggregates connections from a multitude of end-devices, the gateway becomes a focal point of cumulative risk. It inherits the security posture, or lack thereof, of every device connected to it.[8] A single vulnerable smart bulb or fitness tracker can become the weak link that allows an attacker to compromise the gateway, which in turn can serve as a pivot point to attack

the broader internal network.

The strategic importance of the gateway is underscored by the nature of modern BLE attacks. Research by the NCC Group demonstrated a link-layer relay attack that completely bypasses proximity-based authentication, a security measure central to many BLE systems. Using cheap, off-the-shelf hardware, this attack allows a malicious actor to unlock a car or a smart lock from hundreds of miles away by relaying the BLE signals between the legitimate key fob and the device.[3] This type of attack targets the fundamental interaction between devices, a process that is arbitrated and managed by the gateway. The market's relentless drive for user convenience and "quick connect" features, exemplified by the mechanisms exploited by tools like the Flipper Zero, has inadvertently fostered a culture of "security debt".[7] Manufacturers, prioritizing a seamless user experience, often opt for the least secure pairing methods and may not implement robust, layered defenses. This has created a heterogeneous ecosystem where a gateway must manage devices with wildly varying and often unknown levels of security. It cannot inherently trust any single device to be secure, making a proactive, network-wide security monitoring solution essential.

The Gateway as a Strategic Defense Point

Given that the gateway is a natural chokepoint for all local BLE traffic, it represents the most logical and efficient location for deploying a security monitoring solution. An Intrusion Detection System (IDS) placed on the gateway can oversee the entire BLE piconet, analyzing traffic from all connected devices without the need to install software on each individual end-node.[10] This is particularly advantageous in the BLE ecosystem, where many devices are resource-constrained (e.g., running on coin-cell batteries) and lack the computational power, memory, or updatable firmware necessary to host their own security agents.

The evolution of attacks further cements the gateway's role as a critical defensive asset. While early Bluetooth attacks often targeted a single device (e.g., "BlueSnarfing" to steal a contact list), more recent and sophisticated exploits target the network protocol and interactions themselves. Attacks like the link-layer relay [3] and the BLUFFS attack (which compromises future sessions after an initial key negotiation downgrade) [6] are network-level threats. They exploit the trust and communication patterns between devices. Consequently, an effective defense must also operate at the network level. A gateway-based Network Intrusion Detection

System (NIDS) is perfectly positioned to provide this defense, monitoring the collective behavior of the network to identify patterns indicative of such advanced attacks. This transforms the gateway from a simple data forwarder into an intelligent security sentinel for the entire local IoT environment.

Characterizing the Adversarial Landscape for BLE Gateways

To develop an effective intrusion detection system, it is first necessary to define a clear and representative set of threats that the system will be designed to identify. The spectrum of BLE attacks is broad, but for the purposes of a focused and achievable research project, a selection of attacks must be made that are both simple to implement in a testbed environment and exhibit distinct characteristics observable at the gateway.

A Taxonomy of BLE Attacks

Attacks on BLE can be broadly categorized based on the attacker's methodology and objectives. A common taxonomy divides them into three main types [5]:

1. **Passive Attacks:** These involve the attacker listening to communications without actively interfering. The primary examples are **eavesdropping**, where the attacker attempts to capture and decrypt data, and **device tracking**, where advertising packets are used to monitor a user's location, even when features like MAC address randomization are employed.[1]
2. **Active Attacks:** These require the attacker to transmit malicious packets to manipulate the network. This category is diverse and includes:
   - **Man-in-the-Middle (MitM) Attacks:** The attacker positions themself between two communicating devices, relaying traffic while potentially reading or modifying it.[5]
   - **Spoofing/Impersonation:** The attacker mimics a legitimate device by using its MAC address, potentially tricking a central device into connecting with it.[12]
   - **Replay Attacks:** The attacker captures valid data packets and re-transmits them later to gain unauthorized access or repeat an action.[8]
   - **Downgrade Attacks:** As previously discussed, the attacker forces devices to

use weaker security protocols than they are capable of.[1]

3. **Denial of Service (DoS) Attacks:** The attacker's goal is to disrupt or completely disable the communication channel. This is often achieved by flooding the network with traffic, overwhelming the target devices.[9] Examples include **BlueSmacking**, which uses oversized L2CAP packets [14], and the more recent advertising packet flood attacks.

Selected Attacks for This Study

For this project, two attacks have been selected from the active and DoS categories. This choice is based on three primary criteria: implementation simplicity, high impact on the network, and the generation of distinct traffic patterns that can be monitored at the gateway, particularly within the unencrypted advertising phase of communication.

Attack 1: Advertising Packet Flooding (Denial of Service)

- **Description:** This attack involves an adversary using a transmitter to bombard the 2.4GHz spectrum with a high volume of BLE advertising packets. These packets are sent on the three advertising channels (37, 38, and 39) and do not require any prior connection or pairing with a target device. The attack is remarkably easy to execute using cheap, widely available hardware such as a Raspberry Pi or a Flipper Zero device running custom firmware.[3]
- **Impact:** The primary effect of an advertising flood is denial of service. The constant stream of malicious packets can overwhelm the radio and processing capabilities of nearby BLE devices, causing them to become sluggish, crash, or be unable to process legitimate communications. This has been observed in real-world incidents, such as an Android phone controlling a medical insulin pump being crashed by a BLE attack at a conference.[9] For a gateway, this attack can prevent it from discovering and connecting to legitimate peripherals, effectively isolating it from its sensor network.
- **Observability at the Gateway:** From the gateway's perspective, this attack is characterized by a sudden and dramatic increase in the rate of incoming advertising packets. The source MAC addresses of these packets will likely

appear random and change rapidly, as the attacker's tools cycle through different addresses to maximize disruption.

Attack 2: MAC Address Spoofing (Impersonation)

- **Description:** In a MAC address spoofing attack, the adversary first passively scans the network to identify the unique 48-bit Bluetooth Device Address (BD_ADDR), or MAC address, of a trusted peripheral device. The attacker then configures their own device to transmit advertising packets using this stolen MAC address.[12] This impersonation can serve as a foundational step for more complex attacks, such as tricking the gateway into initiating a connection with the malicious device instead of the legitimate one, which can then lead to a full MitM attack or the injection of malicious commands.[14]
- **Impact:** A successful spoofing attack can completely undermine the trust model of a BLE network that relies on MAC address whitelisting for security. It can lead to connection hijacking, data interception, or the delivery of malicious payloads to the gateway. Even if a connection is not established, the presence of two devices advertising with the same MAC address can cause confusion and instability in the gateway's connection management logic.
- **Observability at the Gateway:** Unlike a brute-force flood, a spoofing attack can be more subtle. However, it leaves distinct statistical traces. If the attacker and the legitimate device are in different physical locations, the gateway will receive packets from the same MAC address but with significantly different Received Signal Strength Indicator (RSSI) values. This would result in a bimodal or highly variable RSSI distribution for that MAC address, in contrast to the relatively stable RSSI expected from a stationary legitimate device. Additionally, the timing and sequence of packets from the two sources may be inconsistent, creating detectable anomalies in inter-arrival times.

A crucial aspect justifying the use of machine learning for this project is the inherent asymmetry between the simplicity of launching these attacks and the complexity of defending against them. Open-source scripts and tools available on platforms like GitHub make it trivial for even a novice to launch a DoS flood or a spoofing attack.[15] A human observer could easily recognize the difference between a single device advertising normally and a screen full of random packets from a flood attack. However, programming a traditional, rule-based system to make this distinction

reliably is challenging. A simple rule like

if packet_rate > threshold, then alert is brittle; it cannot easily distinguish a legitimate "crowd" of new devices (e.g., in a public space) from a malicious flood. Nor can it robustly detect a subtle spoofing attack based on RSSI variance without complex statistical modeling. The true novelty and value of this research, therefore, lie not in merely detecting a single, obvious attack, but in constructing a classifier that can learn the nuanced, multi-dimensional statistical differences between multiple network states: Normal, Flood, and Spoof. This elevates the project from a simple security demonstration to a genuine machine learning research problem.

A Framework for Behavioral Fingerprinting of BLE Traffic

To address the challenge of classifying BLE network attacks at the gateway, this study proposes a framework grounded in the principles of anomaly-based intrusion detection. The framework leverages the gateway's strategic position to create "behavioral fingerprints" of network activity by analyzing traffic metadata in real-time. This approach is designed to be lightweight, application-agnostic, and capable of detecting both known and potentially novel attack patterns.

IDS Paradigms: Misuse vs. Anomaly Detection

Intrusion Detection Systems are generally categorized into two main paradigms based on their detection methodology [11]:

1. **Misuse Detection (Signature-Based):** This is the most common approach in commercial IDS products. The system maintains a database of "signatures," which are patterns corresponding to known intrusion techniques or system vulnerabilities. Incoming network traffic is compared against this database, and a match triggers an alert. The primary advantage of misuse detection is its high accuracy and very low false positive rate when identifying known attacks. Its significant drawback, however, is its inability to detect novel, or "zero-day," attacks for which no signature exists.[18] As attackers constantly evolve their methods, a purely signature-based system is always one step behind.

2. **Anomaly Detection (Behavior-Based):** This approach first establishes a baseline model of "normal" system or network behavior. It then continuously monitors the system, and any activity that deviates significantly from this established baseline is flagged as a potential intrusion. The main strength of anomaly detection is its inherent ability to detect new and unforeseen attacks, as it does not rely on prior knowledge of attack patterns.[18] The primary challenge lies in accurately defining the boundary between normal and abnormal behavior, which can sometimes lead to a higher rate of false positives, as not all anomalies are malicious.

For a research project that aims to provide novel contributions and address the dynamic, evolving threat landscape of BLE, an anomaly detection approach is demonstrably superior. It offers the potential to create a more resilient and future-proof security solution that is not constrained by a static library of known threats.[18]

Proposed Architecture: A Gateway-Centric Anomaly-Based NIDS

The architecture proposed in this study is a **Network-based Intrusion Detection System (NIDS)**, as it is designed to monitor network traffic as it flows to and from the gateway, rather than being installed on the individual end-devices (which would be a Host-based IDS, or HIDS).[10] This NIDS will be implemented directly on the BLE gateway, which will function as an intelligent "watchdog" for the local piconet. This concept is adapted from research in BLE mesh networks, where a designated node is tasked with analyzing traffic flow to protect the integrity of the mesh.[21]

The operational logic of the proposed IDS will follow a two-phase process, a standard practice in machine learning system design [21]:

1. **Learning Phase:** In this offline phase, the system is trained. Labeled traffic data, captured from a controlled testbed environment, is used to teach a machine learning classifier to distinguish between different network states. The data will be explicitly labeled as Normal, Flood Attack, or Spoof Attack.
2. **Detection Phase:** Once the model is trained and validated, it is deployed onto the gateway. In this online phase, the IDS actively monitors live BLE advertising traffic, extracts features in real-time, and uses the trained model to classify the current state of the network, raising an alert if an attack is detected.

The Core Concept: Behavioral Fingerprinting through Time-Windowed Analysis

A key innovation of this framework is its focus on analyzing metadata rather than packet payloads. In many BLE communications, the application data payload may be encrypted, making deep packet inspection impossible without compromising the device's keys.[4] Furthermore, even if unencrypted, the payload data is application-specific and would require a different parser for every type of device connected to the gateway. To circumvent these issues, the proposed IDS will operate on easily accessible, unencrypted metadata from the link and physical layers.

The system will analyze traffic in discrete time windows (e.g., 5 or 10 seconds), a technique that has proven effective for identifying traffic patterns in BLE mesh networks.[21] By calculating statistical properties of the traffic within each window, the IDS creates a "behavioral fingerprint"—a quantitative snapshot of the network's behavior at that moment. Different network states will produce measurably different fingerprints:

- **Normal Fingerprint:** Characterized by a relatively low and stable packet rate from a known set of devices. For a stationary device, the RSSI will be stable, and the time between its advertising packets (inter-arrival time) will be consistent.
- **Flood Attack Fingerprint:** Defined by an extremely high total packet rate, a high count of unique source MAC addresses (as the attacker spoofs them), and very low inter-arrival times.
- **Spoof Attack Fingerprint:** Identified by anomalies associated with a specific, legitimate MAC address. This could include a high standard deviation in RSSI (indicating signals are coming from two different locations—the real device and the spoofer) or erratic inter-arrival times that deviate from the legitimate device's known advertising interval.

This approach deliberately leverages what might be considered "dumb" data—signal strength, timestamps, packet counts—to make intelligent security decisions. The underlying principle is that while an attacker might be able to encrypt their malicious payload, they cannot escape the laws of physics and protocol. To achieve their objective, such as overwhelming a device or impersonating it, they must manipulate the physical or link layers by sending more packets, sending them faster, or sending them from a different physical location. These manipulations inevitably leave statistical traces in the traffic metadata, regardless of encryption or application-layer

content.[2] This makes the behavioral fingerprinting approach powerful, as it is inherently lightweight, fast, and, most importantly, agnostic to the specific applications of the devices being protected. A single, well-trained IDS model could therefore protect a gateway managing a diverse array of devices, from medical sensors to smart locks, without needing to understand the unique protocol of each one, rendering the solution highly scalable and broadly applicable.

Novel Feature Engineering from Advertising Packet Metadata

The success of any machine learning-based intrusion detection system is fundamentally dependent on the quality and relevance of the features used for training and classification. Feature engineering—the process of using domain knowledge to extract informative features from raw data—is a critical step that can significantly improve model accuracy, reduce false positives, and provide deeper insights into the problem domain.[22] This section details a novel set of features designed specifically for creating behavioral fingerprints of BLE advertising traffic at a gateway.

The Importance of Feature Engineering

Raw network data, such as a stream of packets, is not in a format that machine learning algorithms can directly process. It must be transformed into a structured feature vector—a numerical representation of the data's key characteristics. The choice of which features to engineer is paramount. Well-designed features can highlight the subtle patterns that differentiate normal behavior from malicious attacks, while poorly chosen or irrelevant features can add noise and degrade the model's performance.[25] The goal of feature engineering in this context is to distill the dynamic, time-series nature of BLE traffic into a fixed-size vector that captures the statistical essence of network behavior within a given time window.

Proposed Feature Set

The proposed feature set is calculated over discrete, non-overlapping time windows (e.g., a window size t=5 seconds). For each window, the gateway's sniffing script will compute a vector of features that describe the advertising traffic observed during that interval. These features are designed to be sensitive to the changes expected during the selected Flood and Spoof attacks and are categorized into four groups.

Flow-Based Features

These features describe the overall traffic flow and composition within the window.

- total_packets: The total count of all advertising packets received in the window. This is expected to be the primary indicator of a Flood attack, which would cause this value to increase dramatically.
- unique_macs: The number of unique source MAC addresses observed in the window. During normal operation with a fixed set of peripherals, this number should be stable. A Flood attack using randomized MAC addresses will cause this to spike.
- new_mac_rate: The number of MAC addresses seen in the current window that were not seen in the previous window. This feature captures the rate of change in the device population and is another strong indicator of a Flood attack.

Temporal Features

These features analyze the timing of packets, calculated on a per-MAC-address basis and then aggregated (e.g., by averaging across all MACs in the window).

- inter_arrival_time_mean: The average time between consecutive packets from the same source MAC address. Legitimate BLE devices advertise at a configured interval, leading to a predictable mean. Attack tools may transmit packets as fast as possible, resulting in a very low mean inter-arrival time.
- inter_arrival_time_stddev: The standard deviation of the inter-arrival times for a given MAC. A stable device should have a low standard deviation. An attacker's transmissions might be less consistent, or network congestion from a Flood

attack could increase the variance for all devices.

Signal Strength Features

These features are derived from the Received Signal Strength Indicator (RSSI), a measure of the power of the incoming signal. They are also calculated per-MAC and then aggregated.

- rssi_mean: The average RSSI value for packets from a specific MAC address. While this can fluctuate, a Spoof attack might be detectable if the attacker's signal is significantly stronger or weaker than the legitimate device's, shifting the mean.
- rssi_stddev: The standard deviation of RSSI values from a single MAC address. This is a crucial feature for detecting Spoof attacks. A stationary legitimate device will have a low RSSI standard deviation. However, if an attacker is spoofing that device's MAC from a different physical location, the gateway will receive packets from the same MAC at two different signal strengths, leading to a high standard deviation.

Packet Structure Features

These features examine the content of the advertising packets without needing to understand the application-layer semantics.

- adv_payload_len_mean: The average length of the advertising data (AD) structure payload. Legitimate devices typically have a fixed payload length. Attack tools may use fixed, minimal, or randomized lengths, which would alter this statistical measure.
- adv_payload_len_stddev: The standard deviation of the advertising payload length. A high standard deviation could indicate an attacker who is randomizing packet contents to evade simple signature-based filters.
- manufacturer_data_entropy: The Shannon entropy calculated on the Manufacturer Specific Data field within the advertising payload. Legitimate devices often include structured, relatively low-entropy data (e.g., identifiers, sensor readings). A flooding tool might use zeroed-out or completely random

(high-entropy) data, creating a detectable difference.

The following table summarizes the proposed feature set, providing a clear blueprint for the data extraction process.

Table 1: Feature Set for BLE Behavioral Fingerprinting

Feature Name	Description	Data Type	Rationale / Expected Attack Indication
Flow-Based Features			
total_packets	Total number of advertising packets in the time window.	Integer	Dramatically increases during a **Flood** attack.
unique_macs	Count of unique source MAC addresses in the time window.	Integer	Increases during a **Flood** attack that uses randomized MACs.
new_mac_rate	Number of newly observed MAC addresses in the window.	Integer	Spikes during the onset of a **Flood** attack.
Temporal Features			
inter_arrival_time_mean	Mean time between packets from the same MAC address.	Float	Decreases significantly for attacking devices during **Flood** or **Spoof** attacks.
inter_arrival_time_stddev	Standard deviation of inter-packet times from the same MAC.	Float	May increase during attacks due to inconsistent transmission or network jitter.
Signal Strength Features			
rssi_mean	Mean RSSI from a single MAC address.	Float	May shift during a **Spoof** attack if the

			attacker is at a different distance.
rssi_stddev	Standard deviation of RSSI from a single MAC address.	Float	Key indicator for **Spoofing**. A high value suggests signals from one MAC are originating from multiple locations.
Packet Structure Features			
adv_payload_len_mean	Mean length of the advertising data payload.	Float	May deviate from normal if an attacker uses malformed or non-standard packets.
adv_payload_len_stddev	Standard deviation of the advertising payload length.	Float	May increase if an attacker is randomizing packet structures.
manufacturer_data_entropy	Shannon entropy of the manufacturer-specific data field.	Float	May be abnormally low (e.g., all zeros) or high (random data) during an attack compared to structured legitimate data.

It is crucial to recognize that no single feature is likely to be a perfect classifier. The true power of this approach comes from the *combination* of these features. For example, a Flood attack is not just characterized by a high total_packets value, but by a combination of high total_packets, high unique_macs, and low inter_arrival_time_mean. A Spoof attack, conversely, might have a normal total_packets value but will be distinguished by a high rssi_stddev for a *specific* MAC address. A machine learning model is uniquely suited for this task because it can learn the complex, non-linear decision boundaries within this multi-dimensional feature space. It can automatically determine the optimal weights and interactions between these features to differentiate between the subtle statistical signatures of normal operation and various attack scenarios—a task that would be exceedingly difficult and brittle to implement using a traditional, hand-crafted set of if-then rules. This inherent

capability to learn from multi-dimensional data is the core justification for applying a machine learning approach to this problem.

A Practical Guide to Testbed Implementation and Data Generation

A cornerstone of credible and impactful cybersecurity research is reproducibility. This section provides a detailed, step-by-step guide for constructing a low-cost hardware testbed and generating a labeled dataset of BLE traffic. This "cookbook" approach is designed to be followed by the student interns, ensuring a consistent and well-documented experimental process.

Hardware and Software Bill of Materials

The testbed is designed to be built with accessible and affordable components, making the research easy to replicate by other academic or industry labs.

- **Hardware:**
  - **3x Raspberry Pi (Model 3B+ or newer):** These single-board computers are powerful enough to act as BLE gateways, peripherals, and attackers. Newer models with improved Bluetooth capabilities are recommended.
  - **3x High-Quality MicroSD Cards (16 GB or larger):** For the operating system and data storage.
  - **3x USB-C Power Supplies:** To provide stable power to the Raspberry Pis.
  - **1x Legitimate BLE Peripheral:** A commercial off-the-shelf device such as a fitness tracker (e.g., Fitbit), a smart tag (e.g., Tile), or a wireless sensor. This will provide a source of realistic, legitimate BLE traffic.
  - **(Optional) 1-2x External USB BLE Dongles:** These can be used on the attacker Pi to provide a more powerful transmitter or to use a different Bluetooth stack than the Pi's built-in one, adding variability to the attack signatures.[27]
- **Software:**
  - **Raspberry Pi OS (formerly Raspbian):** The official Linux-based operating system for the Raspberry Pi.

- **Python 3:** The primary programming language for the data collection and attack scripts.
- **BlueZ Protocol Stack:** The official Linux Bluetooth stack. Tools like bluetoothctl and hcitool will be used for basic configuration and testing.[27]
- **Python Libraries:**
  - **Packet Sniffing/Parsing:** A library like pyshark (a Python wrapper for the powerful tshark command-line tool) or bleson [30] for capturing and parsing BLE packets.
  - **Packet Crafting:** A library like Scapy or the more specialized BLESuite [31] can be used on the attacker Pi to construct and transmit custom advertising packets.
  - **Data Science Stack:** pandas for data manipulation and creating CSV files, numpy for numerical operations, and scikit-learn for machine learning model implementation.

Testbed Setup and Configuration

The three Raspberry Pis will be configured with distinct roles to simulate a complete network environment.

- Step 1: Prepare the Raspberry Pis.
  For each Pi, flash a fresh image of Raspberry Pi OS to a MicroSD card. Boot the Pi, connect it to a network (via Ethernet for stability), and perform standard setup procedures (e.g., update packages, set hostname). Install all necessary software dependencies, including Python 3, pip, and the required development libraries like libbluetooth-dev as detailed in various online tutorials.15
- **Step 2: Designate Roles.**
  - **Pi 1 (Gateway / Sensor):** This Pi's sole function is to listen. Its internal Bluetooth radio will be put into scanning mode to capture all advertising packets in the vicinity. It will run the primary data collection script, which sniffs traffic, calculates features, and logs the data. It should be placed in a central location relative to the other devices.
  - **Pi 2 (Legitimate Peripheral Simulator):** This Pi will be configured to emulate a standard BLE peripheral. Using bluetoothctl commands [32] or a simple Python script, it will be set to broadcast advertising packets with a consistent device name and at a regular, fixed interval (e.g., every 500 ms). This, along with the commercial BLE peripheral, will generate the baseline "Normal" traffic.

- **Pi 3 (Attacker):** This Pi is the adversarial node. It will be equipped with the attack scripts and will be used to launch the Flood and Spoof attacks against the environment. If using an external USB dongle, the Pi's internal Bluetooth should be disabled to avoid interference.

Data Collection Scripts

The core of the data generation process lies in three Python scripts.

- Gateway Sniffing Script (on Pi 1):
  This script will be the most complex. It will use a library like pyshark to start a capture on the BLE interface. The script will be designed to:
  1. Capture raw advertising packets in real-time.
  2. For each captured packet, parse and store its essential metadata: a high-precision timestamp, the source MAC address, the RSSI value, and the raw advertising payload.
  3. Buffer these packets into lists corresponding to fixed time windows (e.g., 5 seconds).
  4. At the conclusion of each time window, process the buffered packets to calculate the full feature vector as defined in Section 4.
  5. Append this feature vector, along with a manually assigned label (Normal, Flood, or Spoof, depending on the active phase of the experiment), as a new row to a CSV file.
- **Attack Scripts (on Pi 3):**
  - **Advertising Flood Script:** This script can be implemented as a simple loop. It will use a library like Scapy or bleson to craft and transmit a high volume of ADV_IND packets. Inside the loop, it should randomize the source MAC address and potentially the payload content for each packet to effectively simulate a "packet storm." Several open-source examples of such flooding scripts exist on GitHub that can be adapted for this purpose.[16]
  - **MAC Spoofing Script:** This script will have two stages. First, it will perform a quick scan to discover the MAC address of the Legitimate Peripheral (either Pi 2 or the commercial device). Then, it will enter a loop where it transmits advertising packets, but with the source MAC address set to the victim's address. Tools like Gattacker and the concepts behind them can serve as a reference for this implementation.[15]

Data Generation Procedure

To create a clean, well-labeled dataset, the data collection should be performed in distinct, timed phases.

- **Phase 1: Baseline "Normal" Traffic (Duration: 15-20 minutes):**
  - Start the Gateway Sniffing Script on Pi 1.
  - Power on the Legitimate Peripheral Simulator (Pi 2) and the commercial BLE peripheral.
  - Allow the system to run undisturbed. All data collected during this phase should be labeled as Normal.
- **Phase 2: "Flood Attack" Traffic (Duration: 15-20 minutes):**
  - While the normal traffic sources are still active, execute the Advertising Flood Script on the Attacker Pi (Pi 3).
  - The Gateway script will continue to capture all traffic. All data collected during this phase should be labeled as Flood.
- **Phase 3: "Spoof Attack" Traffic (Duration: 15-20 minutes):**
  - Stop the flood attack script.
  - Execute the MAC Spoofing Script on the Attacker Pi, targeting the MAC address of the Legitimate Peripheral Simulator (Pi 2).
  - All data collected during this phase should be labeled as Spoof.

This structured procedure will result in a single CSV file containing thousands of labeled feature vectors, ready for the machine learning phase. It is important to recognize that this dataset is not merely a means to an end; it is a significant research deliverable in its own right. The academic community frequently laments the lack of high-quality, publicly available datasets for specific network security scenarios, a challenge noted in BLE mesh research and broader IDS studies.[20] By meticulously documenting the testbed setup and data generation methodology, the interns can prepare the dataset for public release alongside their paper. This act of contributing a new resource to the community significantly increases the potential impact and citation count of their work, transforming a necessary procedural step into a key scientific contribution.

Machine Learning Model Selection for Real-Time Classification

Following the creation of a robust, labeled dataset, the next stage of the project is to select, train, and evaluate machine learning models capable of classifying BLE network behavior in real-time. The choice of models is guided by the project's specific constraints: the need for simplicity of implementation for student interns, low computational overhead suitable for deployment on a gateway device like a Raspberry Pi, and a degree of interpretability to understand the model's decision-making process.

Rationale for Model Selection

The field of intrusion detection has seen the application of a wide range of machine learning algorithms, from classic statistical models to complex deep learning architectures. While deep learning models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks have demonstrated high accuracy in some network intrusion tasks, they come with significant drawbacks for this particular project.[34] They are often treated as "black boxes," making it difficult to understand

why they arrive at a particular conclusion.[33] Furthermore, they are computationally expensive to train and require more resources for inference, which may not be ideal for a resource-constrained gateway.

Therefore, this study focuses on classic, yet powerful, supervised learning algorithms, particularly tree-based ensemble methods. These models offer an excellent balance of performance, efficiency, and interpretability, making them highly suitable for the task at hand.

Candidate Models

Three candidate models have been selected for evaluation. They represent a spectrum of complexity and are well-supported by the scikit-learn library in Python,

simplifying implementation.

1. **Decision Tree (DT):** A Decision Tree is a non-parametric supervised learning method used for classification and regression. It functions by creating a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. The model structure resembles a flowchart, where each internal node represents a test on a feature, each branch represents the outcome of the test, and each leaf node represents a class label.[25] Decision Trees are simple to understand and visualize, making them an excellent baseline model for this project.[25]
2. **Random Forest (RF):** A Random Forest is an ensemble learning method that operates by constructing a multitude of Decision Trees at training time. For a classification task, the final output is the class selected by the most trees (mode). It is a form of "bagging" (bootstrap aggregating) that also introduces randomness in feature selection at each node split. This process generally results in a model with higher accuracy and robustness against overfitting compared to a single Decision Tree.[25] A key advantage of the Random Forest model is its ability to calculate and report "feature importance" scores, which provide a quantitative measure of how much each feature contributed to the classification decision. This is an invaluable tool for analysis and explainability.[23]
3. **Support Vector Machine (SVM):** An SVM is a powerful and versatile classification algorithm that works by finding an optimal hyperplane that best separates data points of different classes in a high-dimensional space. The "support vectors" are the data points that lie closest to the decision boundary, and they are the most critical elements in defining the hyperplane.[25] SVMs can be very effective, particularly for datasets with many features. However, they can be more computationally intensive to train than tree-based models and may require more careful tuning of their hyperparameters (e.g., the kernel type, C, and gamma parameters) to achieve optimal performance.[20]

Training and Evaluation Protocol

A standardized protocol will be used to train and evaluate each of the three models to ensure a fair and direct comparison of their performance.

- **Step 1: Data Preparation.** The CSV dataset generated in Section 5 will be loaded using the pandas library. The feature columns will be separated from the label column (Normal, Flood, Spoof).

- **Step 2: Data Splitting.** The dataset will be split into a training set and a testing set. A standard 80/20 split is recommended, where 80% of the data is used to train the models and the remaining 20% is held back as unseen data for evaluation.
- **Step 3: Model Training.** Each of the three models (DT, RF, SVM) will be instantiated and trained using the fit() method on the training data.
- **Step 4: Model Evaluation.** The performance of each trained model will be rigorously assessed on the unseen testing data. The following standard classification metrics will be calculated:
  - **Confusion Matrix:** A table that visualizes the performance of the classifier by showing the counts of true positives, true negatives, false positives, and false negatives for each class. This is essential for understanding where the model is succeeding and where it is failing.
  - **Accuracy:** The overall proportion of correctly classified instances. While useful, it can be misleading on imbalanced datasets.
  - **Precision, Recall, and F1-Score:** These metrics provide a more nuanced view of performance, especially for multi-class problems. They will be calculated on a per-class basis.
    - *Precision* measures the accuracy of positive predictions (of all instances predicted as Attack, how many actually were?).
    - *Recall* (or Sensitivity) measures the model's ability to find all relevant instances (of all actual Attacks, how many did the model find?).
    - *F1-Score* is the harmonic mean of Precision and Recall, providing a single score that balances both concerns.
  - **Matthews Correlation Coefficient (MCC):** A highly robust statistical metric that produces a high score only if the classifier obtained good results in all four categories of the confusion matrix (true positives, false negatives, true negatives, and false positives). It is particularly well-suited for evaluating binary classifications and is generally regarded as a balanced measure even if the classes are of very different sizes.[37]

The results of this evaluation will be compiled into a comprehensive table to facilitate direct comparison.

Table 2: Comparative Performance of ML Classifiers for BLE Attack Detection

Model	Overall Accuracy	Normal Class (F1-S	Flood Attack	Spoof Attack	Normal Class (Prec	Flood Attack	Spoof Attack	Normal Class (Rec	Flood Attack	Spoof Attack

		core)	(F1-Score)	(F1-Score)	ision)	(Precision)	(Precision)	all)	(Recall)	(Recall)
Decision Tree										
Random Forest										
Support Vector Machine										

This table will serve as the central piece of quantitative evidence for the research project. It will not only determine the best-performing model for the proposed IDS but also provide valuable insights into the relative strengths and weaknesses of each algorithm in the specific context of BLE attack detection.

Analysis, Insights, and Deployment Pathways

The culmination of the experimental work lies in the thorough analysis of the results, the extraction of meaningful security insights, and the consideration of a practical pathway toward real-world deployment. This section interprets the quantitative findings from the machine learning evaluation, connects them back to the project's core hypotheses, and assesses the feasibility of implementing the proposed system on a live gateway.

Performance Analysis

The primary analysis will begin with a comparative review of the results presented in

Table 2. Based on extensive research in the field of network intrusion detection, it is hypothesized that the Random Forest (RF) classifier will exhibit the best overall performance.[33] Its ensemble nature typically provides a significant advantage in terms of accuracy and generalization over a single Decision Tree, and it often outperforms SVMs on tabular data without requiring extensive hyperparameter tuning.

A deeper analysis will involve examining the confusion matrix for each model. This will reveal not just the overall error rates but the specific types of errors being made. For instance, does the model frequently misclassify a Spoof attack as Normal traffic? Or is the primary source of confusion between the two attack types, Flood and Spoof? Identifying these specific failure modes is critical for understanding the model's limitations and for guiding future improvements. For example, if the model struggles to distinguish Spoofing from Normal, it may indicate that the features related to RSSI variance are not as discriminative as hypothesized, or that the spoofing attack was not executed in a way that produced a strong enough statistical signal.

Feature Importance and Explainable AI (XAI)

A significant contribution of this research lies not just in building a classifier, but in understanding *how* it works. This is where the interpretability of the Random Forest model becomes a powerful analytical tool. After training the RF model, its feature_importances_ attribute will be extracted. This provides a score for each feature in the vector, indicating its relative contribution to reducing impurity across all trees in the forest. These scores will be visualized in a bar chart, ranking the features from most to least important.

This analysis serves as a direct test of the project's central hypotheses. For example, the expectation is that:

- The total_packets and unique_macs features will rank very highly as the most important predictors for distinguishing the Flood class.
- The rssi_stddev feature will be a top-ranked predictor for the Spoof class.

Confirming these expectations would provide strong empirical evidence that the behavioral fingerprinting approach is successfully capturing the unique statistical traces of the simulated attacks. This process is a form of Explainable AI (XAI), a field dedicated to making "black box" machine learning models more transparent and trustworthy.[38] By using the feature importance plot, one can explain a classification

decision not as an opaque output from a complex algorithm, but with a clear rationale: "The system flagged this time window as a potential spoofing attack primarily because it observed an abnormally high standard deviation in the signal strength from MAC address XX:XX:XX:XX:XX:XX."

This ability to explain the model's reasoning has profound implications. The model itself transforms into a diagnostic tool for understanding BLE security. The ranked list of features provides a "priority list" for security monitoring. It identifies which metadata channels contain the strongest signals of malicious activity. This knowledge is highly transferable. A developer building a much simpler, non-ML, rule-based IDS for a highly resource-constrained device could use the findings of this research to focus their limited monitoring capabilities on the top 2-3 most important features identified by the Random Forest model. In this way, the research provides value that extends beyond the specific machine learning model it produces, offering fundamental insights into the nature of BLE attack detection.

Deployment Considerations

The final step is to assess the practical feasibility of deploying the trained model on a Raspberry Pi gateway for real-time intrusion detection. The analysis suggests that this is highly plausible.

- **Computational Cost:** The entire detection pipeline is designed to be lightweight. The feature extraction process involves calculating simple statistics (counts, means, standard deviations) over small batches of packets, which is computationally inexpensive. The inference phase—using the trained Random Forest model to make a prediction on a new feature vector—is extremely fast, typically taking microseconds to milliseconds. This low overhead makes it well-suited to run continuously in the background on a device like a Raspberry Pi without significantly impacting its primary gateway functions.
- **From IDS to IDPS:** The deployed system can be elevated from a passive Intrusion Detection System (IDS) to an active Intrusion Detection and *Prevention* System (IDPS).[10] Upon the model classifying a time window as an attack, the gateway could be programmed to take automated defensive actions. For example:
    - **Logging and Alerting:** Generate a detailed log of the event and send an alert to a network administrator.
    - **Blocklisting:** If a Flood or Spoof attack is detected from a set of MAC

addresses, the gateway could temporarily add those addresses to a blocklist, ignoring any further packets from them for a defined period.
    - **Adaptive Scanning:** If an attack is suspected, the gateway could alter its scanning behavior, for instance, by focusing only on known, trusted MAC addresses to conserve resources and maintain connections with legitimate devices.

The combination of high accuracy, low computational cost, and clear pathways to automated prevention makes this proposed framework a viable and promising solution for enhancing the security of real-world BLE gateway deployments.

Conclusion and Avenues for Future Research

This report has outlined a comprehensive and novel framework for the classification of network attacks on a Bluetooth Low Energy gateway. The proposed project is designed to be both academically rigorous, with the potential for publication, and practically achievable within a constrained timeframe by a small team. By moving beyond traditional signature-based methods and focusing on the statistical analysis of traffic metadata, this approach offers a lightweight, application-agnostic, and scalable solution to a growing security problem in the IoT domain.

Summary of Contributions

The primary contributions of the proposed research are fourfold:

1. **A Novel IDS Framework:** It proposes and validates a gateway-centric, anomaly-based NIDS for BLE networks. This "behavioral fingerprinting" approach leverages the gateway's strategic position to monitor the entire local piconet without requiring modifications to resource-constrained end-devices.
2. **A Novel Feature Set:** It defines and evaluates a new set of features derived from the metadata of BLE advertising packets. These features, capturing temporal, signal strength, and packet structure characteristics, are designed to create distinct statistical signatures for normal and malicious traffic.
3. **Machine Learning Validation:** It demonstrates through a practical testbed that

simple, interpretable machine learning models, particularly Random Forest, can be trained to accurately classify multiple, distinct BLE attack types (Flood, Spoof) with high performance. The use of feature importance analysis provides a degree of explainability, bridging the gap between black-box models and understandable security diagnostics.

4. **A Publicly Available Dataset:** A significant byproduct of this research is the creation of a labeled dataset for BLE gateway attacks. Given the noted scarcity of such public resources [21], this dataset represents a valuable contribution to the broader research community, enabling further validation and extension of this work.

Limitations

It is important to acknowledge the limitations of this initial study to maintain scientific transparency. The research is conducted in a controlled laboratory environment using a limited number of devices. The attack taxonomy is intentionally constrained to two specific types for feasibility. The testbed utilizes Raspberry Pis as proxies for commercial devices, which may have different hardware and BLE stack behaviors. These factors mean the results, while promising, represent a proof-of-concept rather than a production-ready system.

Future Research Directions

The successful completion of this project will open up numerous avenues for valuable future research. The framework is designed to be extensible, and the following steps represent a logical progression of this work:

- **Expand the Attack Taxonomy:** The most immediate next step is to incorporate more sophisticated and subtle attacks into the dataset and classification model. This includes implementing **Man-in-the-Middle (MitM) attacks** [5], **replay attacks** [8], and critical **protocol downgrade attacks** like the KNOB and BLUFFS vulnerabilities.[1] This would test the model's ability to detect attacks that manipulate the connection and pairing phases, not just the advertising phase.

- **Transition to Online and Adaptive Learning:** The current proposal relies on an offline training phase. A more advanced system would incorporate online or incremental learning, allowing the model to adapt to new, unseen attack patterns over time without requiring a complete retraining cycle. This is crucial for defending against the polymorphic and constantly evolving nature of cyber threats.[18]
- **Real-World Evaluation and Robustness Testing:** The system should be deployed on actual commercial gateway hardware and tested in a "dirty" real-world RF environment with a wide variety of off-the-shelf IoT devices. This would assess its performance against a greater diversity of legitimate traffic patterns and background noise, providing a true measure of its robustness and generalizability.
- **Develop and Evaluate Prevention Mechanisms:** Building on the IDPS concept, future work should focus on the implementation and rigorous evaluation of automated prevention strategies. This could involve studying the effectiveness of different blocklisting policies, exploring the feasibility of targeted jamming against a confirmed malicious device, or designing intelligent channel-hopping sequences to evade localized flooding attacks.

In conclusion, the proposed research provides a strong foundation for a meaningful contribution to the field of IoT security. It addresses a relevant and pressing problem with a novel yet practical solution, offering a clear and achievable path for the student interns to produce high-quality, publishable work.

Works cited

1. VULNERABILITIES IN BLUETOOTH LOW ENERGY AND HOW THEY CAN BE LEVERAGED TO CAUSE HARM - Tech4Humanity Lab, accessed on July 3, 2025, https://tech4humanitylab.clahs.vt.edu/?p=364
2. Cybersecurity Assessment of the Polar Bluetooth Low Energy Heart-rate Sensor - arXiv, accessed on July 3, 2025, https://arxiv.org/html/2404.16117v1
3. Vulnerabilities found in Bluetooth Low Energy gives hackers access to numerous devices, accessed on July 3, 2025, https://www.techrepublic.com/article/vulnerabilities-found-in-bluetooth-low-energy-gives-hackers-access-to-numerous-devices/
4. On the Security of Bluetooth Low Energy in Two Consumer Wearable Heart Rate Monitors/Sensing Devices - PMC, accessed on July 3, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC8839540/
5. Vulnerabilities and Attacks on Bluetooth LE Devices—Reviewing Recent Info - Technical Articles - All About Circuits, accessed on July 3, 2025, https://www.allaboutcircuits.com/technical-articles/vulnerabilities-and-attacks-on-bluetooth-le-devicesreviewing-recent-info/

6. Top 6 Bluetooth Vulnerabilities - Thyrasec, accessed on July 3, 2025, https://www.thyrasec.com/blog/top-6-bluetooth-vulnerabilities/
7. Reviewing BLE and Bluetooth Vulnerabilities and Attacks of 2023 - Thyrasec, accessed on July 3, 2025, https://www.thyrasec.com/blog/reviewing-ble-and-bluetooth-vulnerabilities-and-attacks-of-2023/
8. Top BLE Cybersecurity Vulnerabilities - Blue Goat Cyber, accessed on July 3, 2025, https://bluegoatcyber.com/blog/top-ble-cybersecurity-vulnerabilities/
9. Bluetooth Low Energy denial of service attacks - Wikipedia, accessed on July 3, 2025, https://en.wikipedia.org/wiki/Bluetooth_Low_Energy_denial_of_service_attacks
10. Intrusion Detection and Prevention Systems and Techniques - American Military University, accessed on July 3, 2025, https://www.amu.apus.edu/area-of-study/information-technology/resources/intrusion-detection-and-prevention-systems-and-techniques/
11. Intrusion Detection Systems - CERIAS, Purdue - Purdue University, accessed on July 3, 2025, https://www.cerias.purdue.edu/about/history/coast_resources/idcontent/ids.html
12. 22 common Bluetooth attacks and mitigation techniques - RoboticsBiz, accessed on July 3, 2025, https://roboticsbiz.com/22-common-bluetooth-attacks-and-mitigation-techniques/
13. Bluetooth MAC address spoofing - Flipper Forum, accessed on July 3, 2025, https://forum.flipper.net/t/bluetooth-mac-address-spoofing/23694
14. BLE / NFC Threats - Axelarator, accessed on July 3, 2025, https://axelarator.github.io/posts/blenfc/
15. Machine-in-the-Middle (MitM) BLE Attack - Black Hills Information ..., accessed on July 3, 2025, https://www.blackhillsinfosec.com/machine-in-the-middle-mitm-ble-attack/
16. gmelodie/badblue: Annoying bluetooth DoS "jammer" - GitHub, accessed on July 3, 2025, https://github.com/gmelodie/badblue
17. kimbo/l2ping-flood: Python script to perform l2ping flood ... - GitHub, accessed on July 3, 2025, https://github.com/kimbo/l2ping-flood
18. The Use of Computational Intelligence in Intrusion Detection Systems: A Review - Memorial University of Newfoundland, accessed on July 3, 2025, https://www.mun.ca/computerscience/media/production/memorial/academic/faculty-of-science/computer-science/media-library/research/MUN-CS-2008-05.pdf
19. Intrusion Detection System-Based Network Behavior Analysis: A Systemic Literature Review - The Science and Information (SAI) Organization, accessed on July 3, 2025, https://thesai.org/Downloads/Volume16No3/Paper_78-Intrusion_Detection_System_Based_Network_Behavior_Analysis.pdf
20. (PDF) A Machine Learning-Driven Cloud Intrusion Detection: Impact of Feature Engineering and Dimensionality Reduction on Classification and Optimization - ResearchGate, accessed on July 3, 2025,

https://www.researchgate.net/publication/391717486_A_Machine_Learning-Driven_Cloud_Intrusion_Detection_Impact_of_Feature_Engineering_and_Dimensionality_Reduction_on_Classification_and_Optimization
21. Intrusion Detection System for Bluetooth Mesh Networks: Data ..., accessed on July 3, 2025, https://ece.northeastern.edu/wineslab/papers/andrea/LacavaPercom21.pdf
22. The Improved Network Intrusion Detection Techniques Using the Feature Engineering Approach with Boosting Classifiers - MDPI, accessed on July 3, 2025, https://www.mdpi.com/2227-7390/12/24/3909
23. Machine Learning-Based Network Intrusion Detection for IOT and Smart Detection Using Recursive Feature Elimination, Binning Technique and Grid Search CV - ResearchGate, accessed on July 3, 2025, https://www.researchgate.net/publication/387286580_Machine_Learning-Based_Network_Intrusion_Detection_for_IOT_and_Smart_Detection_Using_Recursive_Feature_Elimination_Binning_Technique_and_Grid_Search_CV
24. A Review on Feature Selection and Ensemble Techniques for Intrusion Detection System, accessed on July 3, 2025, https://www.researchgate.net/publication/352080880_A_Review_on_Feature_Selection_and_Ensemble_Techniques_for_Intrusion_Detection_System
25. Explainable Artificial Intelligence for Intrusion Detection System - MDPI, accessed on July 3, 2025, https://www.mdpi.com/2079-9292/11/19/3079
26. Feature Engineering Methods in Intrusion Detection System: A Performance Evaluation, accessed on July 3, 2025, https://www.ije.ir/article_170882.html
27. Control Bluetooth LE Devices From a Raspberry Pi : 9 Steps - Instructables, accessed on July 3, 2025, https://www.instructables.com/Control-Bluetooth-LE-Devices-From-A-Raspberry-Pi/
28. Using BLE Devices with a Raspberry Pi - Argenox, accessed on July 3, 2025, https://argenox.com/library/bluetooth-low-energy/using-raspberry-pi-ble
29. Bluetooth Low Energy: Read-Write to Sensor Network From Raspberry Pi - Instructables, accessed on July 3, 2025, https://www.instructables.com/Bluetooth-Low-Energy-Read-Write-to-Sensor-Network-/
30. bluetooth lowenergy - BLE - Decoding Advertisement with Python - Stack Overflow, accessed on July 3, 2025, https://stackoverflow.com/questions/75607626/ble-decoding-advertisement-with-python
31. BLESuite is a Python package that provides an easier way to test Bluetooth Low Energy (BLE) device - GitHub, accessed on July 3, 2025, https://github.com/nccgroup/BLESuite
32. Connect to BLE on Raspberry Pi 4 - AVH Support Center, accessed on July 3, 2025, https://support.avh.corellium.com/devices/rpi4/rpi4-ble
33. Comparison of Machine Learning and Deep Learning Models for Network Intrusion Detection Systems - MDPI, accessed on July 3, 2025, https://www.mdpi.com/1999-5903/12/10/167

34. Deep Learning vs. Machine Learning for Intrusion Detection in ..., accessed on July 3, 2025, https://www.mdpi.com/2076-3417/15/4/1903
35. A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks, accessed on July 3, 2025, https://www.researchgate.net/publication/320366926_A_Deep_Learning_Approach_for_Intrusion_Detection_Using_Recurrent_Neural_Networks
36. Comparing a Hybrid Multi-layered Machine Learning Intrusion Detection System to Single-layered and Deep Learning Models, accessed on July 3, 2025, https://repository.stcloudstate.edu/cgi/viewcontent.cgi?article=1190&context=msia_etds
37. Identifying Differences in the Performance of Machine Learning Models for Off-Targets Trained on Publicly Available and Proprietary Data Sets - PMC, accessed on July 3, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC10445286/
38. Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability - Frontiers, accessed on July 3, 2025, https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741/full
39. Use of Explainable Artificial Intelligence for Analyzing and Explaining Intrusion Detection Systems - MDPI, accessed on July 3, 2025, https://www.mdpi.com/2073-431X/14/5/160